

SOLUTIONS – OOADI

LECTURE 10

1. If I look at my TrainClientNetworked.java then I can identify a few issues

- a) Number Format Crash. When asking for number of wagons, if the user types 'three' (and not '3') or leaves an accidental space, then the client crashes.

Number of wagons: three

```
Exception in thread "main" java.lang.NumberFormatException: For input string: "three"
at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
at java.base/java.lang.Integer.parseInt(Integer.java:565)
at java.base/java.lang.Integer.parseInt(Integer.java:662)
at lectuer9Client.TrainClientNetworked.buildAddCommand(TrainClientNetworked.java:173)
at lectuer9Client.TrainClientNetworked.main(TrainClientNetworked.java:83)
```

The same will happen if the client is started (args) using chars for port number

- b) If the client is started with a wrong port number, then it will freeze-up waiting for a connection and the 'quit' command is never reached
- c) No wagon data validation. Wagon length and weight are read as raw strings with zero validation before being sent to the server. Negative numbers, zero values, or text input like "heavy" are all accepted and sent — the server must deal with whatever garbage it receives.
- d) No socket timeout. If the server stops responding mid-conversation (hangs, overloaded), the client will wait forever with no way to recover, forcing the user to manually kill the process. This could be fixed with a time out of for instance 5 seconds using `socket.setSoTimeout(5000);`

2. Looking at the TrainServer.java I can think of the following

- a) There is support for one client only The server accepts exactly one client and when that client disconnects, the server shuts down entirely. There is no loop to accept more clients. A second client trying to connect would wait forever.
- b) Client socket never closed properly. For the client side I have all resource opening inside the try(), providing the try-with-resources approach where automatic closing is ensured. This is not the case here, and you will find my `clientSocket.close()` is in the 'quit' case only. This is fine if things run smoothly and if the client closes. In case of server crash the socket is never closed.
- c) No logging of errors to client. If the server crashes with an `IOException`, it just prints to the server console and exits silently. The connected client receives no notification — it will just hang waiting for a response that never comes, with no idea what went wrong.
-

-
- 3.
- Instead of dying, on client disconnect, the server should log the disconnect and wait for the next client, thereby keeping the service alive. This could be handled using something like the following

```
while (true) {  
    try {  
        Socket clientSocket = serverSocket.accept();  
        System.out.println("Client connected: " + clientSocket.getInetAddress());  
        handleClient(clientSocket, manager); // handle in separate method  
    } catch (IOException e) {  
        System.err.println("Failed to accept client, continuing..."); // log and keep going  
    }  
}
```

- If we were to extend the functionality to more clients, we should have each client run its own thread.

```
while (true) {  
    Socket clientSocket = serverSocket.accept();  
    Thread clientThread = new Thread(() -> {  
        try {  
            handleClient(clientSocket, manager);  
        } catch (IOException e) {  
            System.err.println("Client error: " + e.getMessage());  
        }  
    });  
    clientThread.start(); // each client gets its own thread  
}
```

- The add functionality was added by a blind alcoholic. It is possible to add for instance -1 as the length of a wagon, which corrupts data. Also, it is possible to add weight as 'heavy' and still have the message transmitted. The latter should result in a situation as follows.

```
Command: add  
Train ID: D-100  
Type (DIESEL/ELECTRIC): DIESEL  
Number of wagons: 2  
Wagon 1 length (m): 44  
Wagon 1 weight (t): heavy  
Wagon 2 length (m): 4  
Wagon 2 weight (t): 33  
ERROR bad add: For input string: "heavy"
```

This 'heavy' still ends up in the train list and searching for Diesel trains subsequently will also fail with a ERROR bad add: For input string: "heavy" message.

-
4. The big question is here whether it really is a fault or not. We should use exceptions only when the user would have expected something else than what is found/happening.

- a) Some construction like the following could help address the issue with searching for non-existing trains. This will allow us to throw our own custom exception.

```
public class TrainNotFoundException extends Exception {  
    public TrainNotFoundException(String id) {  
        super("Train not found with ID: " + id);  
    }  
}
```

- b) This one needs to be addressed on the server side. We need a custom exception again, and an arrangement to handle this limitation.

```
public class TooManyResultsException extends Exception {  
    private final int limit;  
    private final int found;  
  
    public TooManyResultsException(int found, int limit) {  
        super("Query returned " + found + " results, maximum allowed is " + limit);  
        this.limit = limit;  
        this.found = found;  
    }  
  
    public int getFound() { return found; }  
    public int getLimit() { return limit; }  
}
```

With this on the table we can try to build the frame work for a new search case where we include both the issue with a train not found and the limit to data being transmitted.

```
private static final int MAX_RESULTS = 100;  
  
case "search" -> {  
    try {  
        DriveType dt = DriveType.valueOf(parts[1].toUpperCase());  
        List<Train> results = manager.search(  
            t -> t.getDriveType() == dt);  
  
        if (results.isEmpty()) {  
            out.println("No trains found of type: " + parts[1]);  
        } else if (results.size() > MAX_RESULTS) {  
            // Don't crash — send partial results with a warning  
            throw new TooManyResultsException(results.size(), MAX_RESULTS);  
        } else {  
            results.forEach(out::println);  
        }  
    }  
    catch (TooManyResultsException e) {
```

```
    out.println("WARNING: " + e.getMessage());
    out.println("Showing first " + e.getLimit() + " results only.");
    // send partial results anyway
    results.stream()
        .limit(MAX_RESULTS)
        .forEach(out::println);

} catch (IllegalArgumentException e) {
    out.println("ERROR unknown type: " + parts[1]);
}
out.println("END");
}
```
